



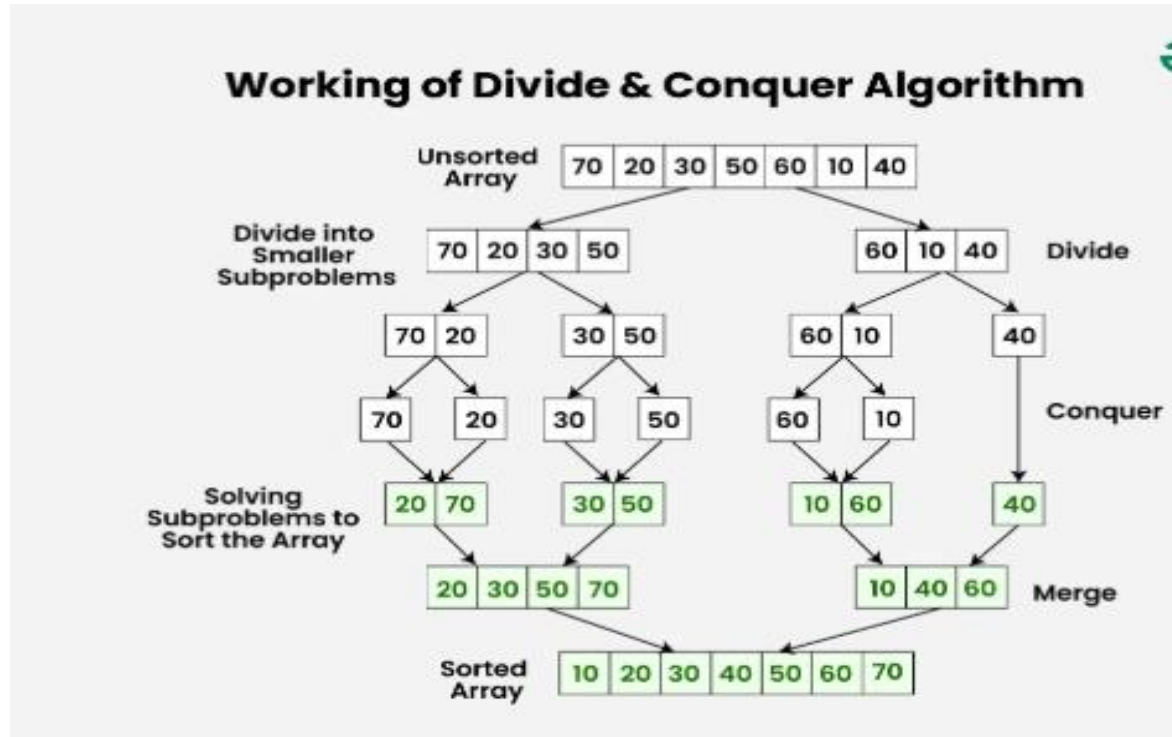
**Bharat Ratna Indira Gandhi College of
Engineering, Solapur
Master Of Computer Application**

**Subject: Data Structure and Algorithms
(MCAC102)**

**Semester-I
Unit 5: Types of Algorithms**

➤ Divide and Conquer:

Divide and Conquer Algorithm is a problem-solving technique used to solve problems by dividing the main problem into subproblems, solving them individually and then merging them to find solution to the original problem. In this article, we are going to discuss how Divide and Conquer Algorithm is helpful and how we can use it to solve problems.



1. Divide:

- Break down the original problem into smaller subproblems.
- Each subproblem should represent a part of the overall problem.
- The goal is to divide the problem until no further division is possible.

2. Conquer:

- Solve each of the smaller subproblems individually.
- If a subproblem is small enough (often referred to as the “base case”), we solve it directly without further recursion.
- The goal is to find solutions for these subproblems independently.

3. Merge:

- Combine the sub-problems to get the final solution of the whole problem.

- Once the smaller subproblems are solved, we recursively combine their solutions to get the solution of larger problem.
- The goal is to formulate a solution for the original problem by merging the results from the subproblems.

❖ **Characteristics of Divide and Conquer Algorithm:**

Divide and Conquer Algorithm involves breaking down a problem into smaller, more manageable parts, solving each part individually, and then combining the solutions to solve the original problem. The characteristics of Divide and Conquer Algorithm are:

- **Dividing the Problem:** The first step is to break the problem into smaller, more manageable subproblems. This division can be done recursively until the subproblems become simple enough to solve directly.
- **Independence of Subproblems:** Each subproblem should be independent of the others, meaning that solving one subproblem does not depend on the solution of another. This allows for parallel processing or concurrent execution of subproblems, which can lead to efficiency gains.
- **Conquering Each Subproblem:** Once divided, the subproblems are solved individually. This may involve applying the same divide and conquer approach recursively until the subproblems become simple enough to solve directly, or it may involve applying a different algorithm or technique.
- **Combining Solutions:** After solving the subproblems, their solutions are combined to obtain the solution to the original problem. This combination step should be relatively efficient and straightforward, as the solutions to the subproblems should be designed to fit together seamlessly.

❖ **Advantages of Divide and Conquer Algorithm:**

- **Solving difficult problems:** Divide and conquer technique is a tool for solving difficult problems conceptually. e.g. Tower of Hanoi puzzle. It requires a way of breaking the problem into sub-problems, and solving all of them as an individual cases and then combining sub- problems to the original problem.

- **Algorithm efficiency:** The divide-and-conquer algorithm often helps in the discovery of efficient algorithms. It is the key to algorithms like Quick Sort and Merge Sort, and fast Fourier transforms.
- **Parallelism:** Normally Divide and Conquer algorithms are used in multi-processor machines having shared-memory systems where the communication of data between processors does not need to be planned in advance, because distinct sub-problems can be executed on different processors.
- **Memory access:** These algorithms naturally make an efficient use of memory caches. Since the subproblems are small enough to be solved in cache without using the main memory that is slower one. Any algorithm that uses cache efficiently is called cache oblivious.

❖ **Disadvantages of Divide and Conquer Algorithm:**

- **Overhead:** The process of dividing the problem into subproblems and then combining the solutions can require additional time and resources. This overhead can be significant for problems that are already relatively small or that have a simple solution.
- **Complexity:** Dividing a problem into smaller subproblems can increase the complexity of the overall solution. This is particularly true when the subproblems are interdependent and must be solved in a specific order.
- **Difficulty of implementation:** Some problems are difficult to divide into smaller subproblems or require a complex algorithm to do so. In these cases, it can be challenging to implement a divide and conquer solution.
- **Memory limitations:** When working with large data sets, the memory requirements for storing the intermediate results of the subproblems can become a limiting factor.

❖ A classic example of a Divide and Conquer algorithm is **Merge Sort**.

Merge Sort Algorithm

Merge Sort divides an array into two halves, sorts each half recursively, and then merges the sorted halves.

DSA-Unit-V

```
#include <iostream>
#include <vector>
using namespace std;
// Merge two sorted halves into a single sorted array
void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1; // Size of left half
    int n2 = right - mid;    // Size of right half
    // Create temporary arrays
    vector<int> leftArr(n1), rightArr(n2);
    // Copy data to temporary arrays
    for (int i = 0; i < n1; i++)
        leftArr[i] = arr[left + i];
    for (int i = 0; i < n2; i++)
        rightArr[i] = arr[mid + 1 + i];
    // Merge the temporary arrays back into arr
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (leftArr[i] <= rightArr[j]) {
            arr[k] = leftArr[i];
            i++;
        } else {
            arr[k] = rightArr[j];
            j++;
        }
        k++;
    }
    // Copy remaining elements of leftArr, if any
    while (i < n1) {
        arr[k] = leftArr[i];
        i++;
        k++;
    }
```

```
    }  
    // Copy remaining elements of rightArr, if any  
    while (j < n2) {  
        arr[k] = rightArr[j];  
        j++;  
        k++;  
    }  
}  
  
// Divide and Conquer function for merge sort  
void mergeSort(vector<int>& arr, int left, int right) {  
    if (left < right) {  
        // Find the middle point  
        int mid = left + (right - left) / 2;  
        // Sort the left and right halves  
        mergeSort(arr, left, mid);  
        mergeSort(arr, mid + 1, right);  
        // Merge the sorted halves  
        merge(arr, left, mid, right);  
    }  
}  
  
int main() {  
    vector<int> arr = {38, 27, 43, 3, 9, 82, 10};  
    cout << "Original array: ";  
    for (int num : arr)  
        cout << num << " ";  
    cout << endl;  
    mergeSort(arr, 0, arr.size() - 1);  
    cout << "Sorted array: ";  
    for (int num : arr)  
        cout << num << " ";  
    cout << endl;
```

```
    return 0;  
}
```

Input:

arr = {38, 27, 43, 3, 9, 82, 10}

Output:

Original array: 38 27 43 3 9 82 10

Sorted array: 3 9 10 27 38 43 82

➤ Strassen's Matrix Multiplication:

Strassen's Matrix Multiplication is an algorithm used to multiply two matrices more efficiently than the standard matrix multiplication method. It uses the **divide and conquer** approach to reduce the number of multiplications required.

Algorithm Overview

For two $n \times n$ matrices A and B, the algorithm:

1. **Divides** each matrix into four submatrices of size $n/2 \times n/2$.
2. **Conquers** by computing intermediate matrices (seven multiplications instead of eight).
3. **Combines** the intermediate results to produce the final product matrix.

The **seven multiplications** (P1 to P7) are:

$$P1 = A11 \cdot (B12 - B22),$$

$$P2 = (A11 + A12) \cdot B22,$$

$$P3 = (A21 + A22) \cdot B11,$$

$$P4 = A22 \cdot (B21 - B11),$$

$$P5 = (A11 + A22) \cdot (B11 + B22),$$

$$P6 = (A12 - A22) \cdot (B21 + B22),$$

$$P7 = (A11 - A21) \cdot (B11 + B12)$$

The resulting submatrices are:

$$C11 = P5 + P4 - P2 + P6, \quad C12 = P1 + P2,$$

$$C21 = P3 + P4, \quad C22 = P5 + P1 - P3 - P7.$$

```
#include <iostream>
using namespace std;
// Function to merge two halves
void merge(int arr[], int left, int mid, int right) {
    int n1 = mid - left + 1; // Size of the left half
    int n2 = right - mid;    // Size of the right half
    int L[n1], R[n2]; // Temporary arrays for left and right halves
    // Copy data to temp arrays L[] and R[]
    for (int i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (int j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];
    int i = 0, j = 0, k = left; // Initial indexes of L[], R[], and merged array
    // Merge the temp arrays back into arr[left...right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
    // Copy remaining elements of L[] if any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    // Copy remaining elements of R[] if any
```



```
        while (j < n2) {
            arr[k] = R[j];
            j++;
            k++;
        }
    }

// Function to implement merge sort
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        // Recursively divide the array
        mergeSort(arr, left, mid);    // Sort the first half
        mergeSort(arr, mid + 1, right); // Sort the second half
        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

// Utility function to print the array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        cout << arr[i] << " ";
    cout << endl;
}

// Main function
int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int arrSize = sizeof(arr) / sizeof(arr[0]);
    cout << "Original array: ";
    printArray(arr, arrSize);
    mergeSort(arr, 0, arrSize - 1);
    cout << "Sorted array: ";
```

```
        printArray(arr, arrSize);  
        return 0;  
    }
```

Input:

arr[] = {12, 11, 13, 5, 6, 7}

Output:

Original array: 12 11 13 5 6 7

Sorted array: 5 6 7 11 12 13

➤ Backtracking:

Backtracking is like trying different paths, and when you hit a dead end, you backtrack to the last choice and try a different route. In this article, we'll explore the basics of backtracking, how it works, and how it can help solve all sorts of challenging problems. It's like a method for finding the right way through a complex choice.

Backtracking is a problem-solving algorithmic technique that involves finding a solution incrementally by trying different options and undoing them if they lead to a dead end. It is commonly used in situations where you need to explore multiple possibilities to solve a problem, like searching for a path in a maze or solving puzzles like Sudoku. When a dead end is reached, the algorithm backtracks to the previous decision point and explores a different path until a solution is found or all possibilities have been exhausted.

❖ Basic Terminologies:

- **Candidate:** A candidate is a potential choice or element that can be added to the current solution.
- **Solution:** The solution is a valid and complete configuration that satisfies all problem constraints.
- **Partial Solution:** A partial solution is an intermediate or incomplete configuration being constructed during the backtracking process.
- **Decision Space:** The decision space is the set of all possible candidates or choices at each decision point.

- **Decision Point:** A decision point is a specific step in the algorithm where a candidate is chosen and added to the partial solution.
- **Feasible Solution:** A feasible solution is a partial or complete solution that adheres to all constraints.
- **Dead End:** A dead end occurs when a partial solution cannot be extended without violating constraints.
- **Backtrack:** Backtracking involves undoing previous decisions and returning to a prior decision point.
- **Search Space:** The search space includes all possible combinations of candidates and choices.
- **Optimal Solution:** In optimization problems, the optimal solution is the best possible solution.

Algorithm

Following is the algorithm for backtracking –

```
1. Start
2. if current_position is goal, return success.
3. else
4. if current_position is an end point, return failed.
5. else-if current_position is not end point, explore and repeat above steps.
6. Stop
```

❖ Example:

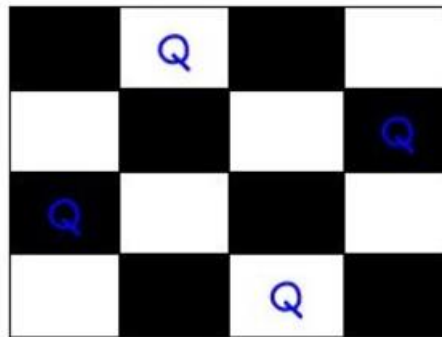
1. What is N Queen Problem?

In **N-Queen problem**, we are given an **N x N** chessboard and we have to place **N** number of queens on the board in such a way that no two queens attack each other. A queen will attack another queen if it is placed in horizontal, vertical or diagonal points in its way. The most popular approach for solving the N Queen puzzle is Backtracking.

Input Output Scenario

DSA-Unit-V

Suppose the given chessboard is of size 4x4 and we have to arrange exactly 4 queens in it. The solution arrangement is shown in the figure below –



The final solution matrix will be –

0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

Backtracking Approach to solve N Queens Problem

In the naive method to solve n queen problem, the algorithm generates all possible solutions. Then, it explores all of the solutions one by one. If a generated solution satisfies the constraint of the problem, it prints that solution.

Follow the below steps to solve n queen problem using the backtracking approach –

- i. Place the first queen in the top-left cell of the chessboard.
- ii. After placing a queen in the first cell, mark the position as a part of the solution and then recursively check if this will lead to a solution.
- iii. Now, if placing the queen doesn't lead to a solution. Then go to the first step and place queens in other cells. Repeat until all cells are tried.
- iv. If placing queen returns a lead to solution return TRUE.
- v. If all queens are placed return TRUE.
- vi. If all rows are tried and no solution is found, return FALSE.

```
#include<iostream>
```

```
using namespace std;
```

DSA-Unit-V

```
#define BOARD_SIZE 5

void displayChess(int chBoard[BOARD_SIZE][BOARD_SIZE]) {
    for (int row = 0; row < BOARD_SIZE; row++) {
        for (int col = 0; col < BOARD_SIZE; col++)
            cout << chBoard[row][col] << " ";
        cout << endl;
    }
}

bool isQueenPlaceValid(int chBoard[BOARD_SIZE][BOARD_SIZE], int crntRow, int
crntCol) {
    // checking if queen is in the left or not
    for (int i = 0; i < crntCol; i++)
        if (chBoard[crntRow][i])
            return false;
    for (int i = crntRow, j = crntCol; i >= 0 && j >= 0; i--, j--)
        //checking if queen is in the left upper diagonal or not
        if (chBoard[i][j])
            return false;
    for (int i = crntRow, j = crntCol; j >= 0 && i < BOARD_SIZE; i++, j--)
        //checking if queen is in the left lower diagonal or not
        if (chBoard[i][j])
            return false;
    return true;
}

bool solveProblem(int chBoard[BOARD_SIZE][BOARD_SIZE], int crntCol) {
    //when N queens are placed successfully
    if (crntCol >= BOARD_SIZE)
        return true;
    // checking placement of queen is possible or not
    for (int i = 0; i < BOARD_SIZE; i++) {
        if (isQueenPlaceValid(chBoard, i, crntCol)) {
```

DSA-Unit-V

```
//if validate, place the queen at place (i, col)
chBoard[i][crntCol] = 1;
//Go for the other columns recursively
if (solveProblem(chBoard, crntCol + 1))
    return true;
//When no place is vacant remove that queen
chBoard[i][crntCol] = 0;
}
}
return false;
}

bool displaySolution() {
    int chBoard[BOARD_SIZE][BOARD_SIZE];
    for(int i = 0; i < BOARD_SIZE; i++)
        for(int j = 0; j < BOARD_SIZE; j++)
            //set all elements to 0
            chBoard[i][j] = 0;
    //starting from 0th column
    if (solveProblem(chBoard, 0) == false) {
        cout << "Solution does not exist";
        return false;
    }
    displayChess(chBoard);
    return true;
}

int main() {
    displaySolution();
}
```

Output

1 0 0 0 0

0 0 0 1 0

0 1 0 0 0

0 0 0 0 1

0 0 1 0 0

2. Subset Sum Problem:

In the sum of subsets problem, there is a given set with some non-negative integer elements. And another sum value is also provided, our task is to find all possible subsets of the given set whose sum is the same as the given sum value.

Input Output Scenario

Suppose the given set and sum value is –

Set = {1, 9, 7, 5, 18, 12, 20, 15}

sum value = 35

All possible subsets of the given set, where sum of each element for every subset is the same as the given sum value are given below –

{1 9 7 18}

{1 9 5 20}

{5 18 12}

Follow the below steps to solve subset sum problem using the backtracking approach –

- i. First, take an empty subset.
- ii. Include the next element, which is at index 0 to the empty set.
- iii. If the subset is equal to the sum value, mark it as a part of the solution.

DSA-Unit-V

- iv. If the subset is not a solution and it is less than the sum value, add next element to the subset until a valid solution is found.
- v. Now, move to the next element in the set and check for another solution until all combinations have been tried.

```
#include <iostream>
```

```
using namespace std;
```

```
void displaySubset(int subSet[], int size) {
```

```
    for(int i = 0; i < size; i++) {
```

```
        cout << subSet[i] << " ";
```

```
    }
```

```
    cout << endl;
```

```
}
```

```
void subsetSum(int set[], int subSet[], int n, int subSize, int total, int nodeCount, int sum) {
```

```
    if( total == sum) {
```

```
        //print the subset
```

```
        displaySubset(subSet, subSize);
```

```
        //for other subsets
```

```
        subsetSum(set, subSet, n, subSize-1, total-set[nodeCount], nodeCount+1,sum);
```

```
        return;
```

```
    }else {
```

```
        //find node along breadth
```

```
        for( int i = nodeCount; i < n; i++ ) {
```

```
            subSet[subSize] = set[i];
```


DSA-Unit-V

```
//do for next node in depth

subsetSum(set, subSet, n, subSize+1, total+set[i], i+1, sum);

}

}

}

void findSubset(int set[], int size, int sum) {

    //create subset array to pass parameter of subsetSum

    int *subSet = new int[size];

    subsetSum(set, subSet, size, 0, 0, 0, sum);

    delete[] subSet;

}

int main() {

    int weights[] = {1, 9, 7, 5, 18, 12, 20, 15};

    int size = 7;

    findSubset(weights, size, 35);

}
```

Output

1 9 7 18

1 9 5 20

5 18 12

➤ **Branch and Bound:**

The **Branch and Bound Algorithm** is a method used in **combinatorial** optimization problems to systematically search for the best solution. It works by dividing the problem into smaller

subproblems, or branches, and then eliminating certain branches based on bounds on the optimal solution. This process continues until the best solution is found or all branches have been explored. **Branch and Bound** is commonly used in problems like the **traveling salesman** and **job scheduling**.

When Branch and Bound Algorithm is used?

The branch and bound algorithm can be used in the following scenario –

- Whenever we encounter an optimization problem whose variables belong to a discrete set. These types of problems are known as discrete optimization problems.
- As discussed earlier, this algorithm is also used to solve combinatorial optimization problem.
- If the given problem is a mathematical optimization problem, then the branch and bound algorithm can also be applied.

Advantages of Branch and Bound Algorithm

The advantages of the branch and bound algorithm are as follows –

- It can reduce the time complexity by avoiding unnecessary exploration of the state space tree.
- It has different search techniques that can be used for different types of problems and preferences.

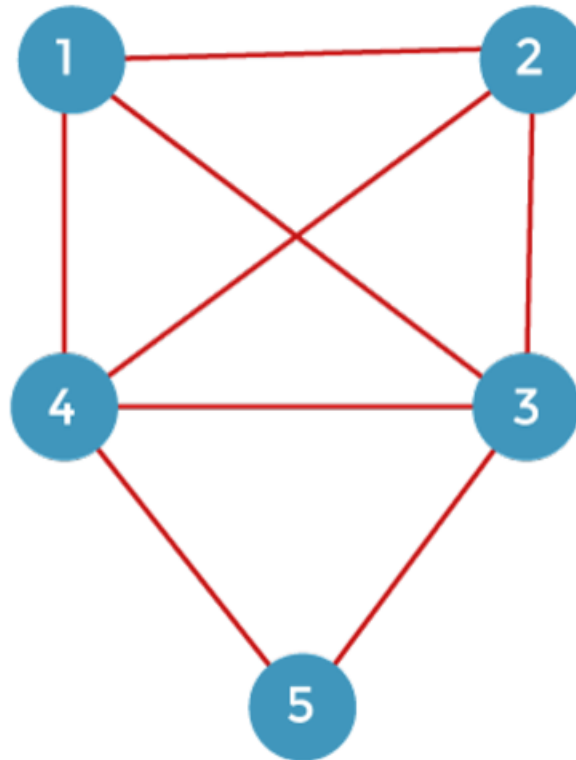
Disadvantages of Branch and Bound Algorithm

Some of the disadvantages of the branch and bound algorithm are as follows –

- In the worst-case scenario, it may search for all the combinations to produce solutions.
- It can time consuming if the state space tree is too large.

➤ **Traveling Salesperson problem using branch and bound:**

- Given the vertices, the problem here is that we have to travel each vertex exactly once and reach back to the starting point. Consider the below graph

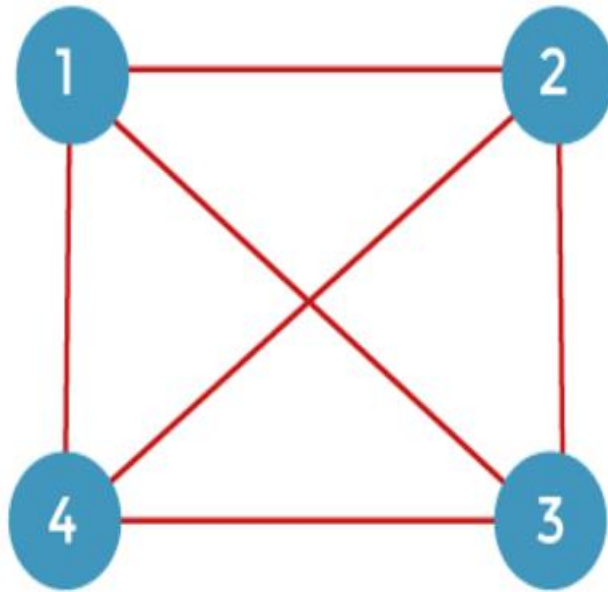


The adjacent matrix of the problem is given below:

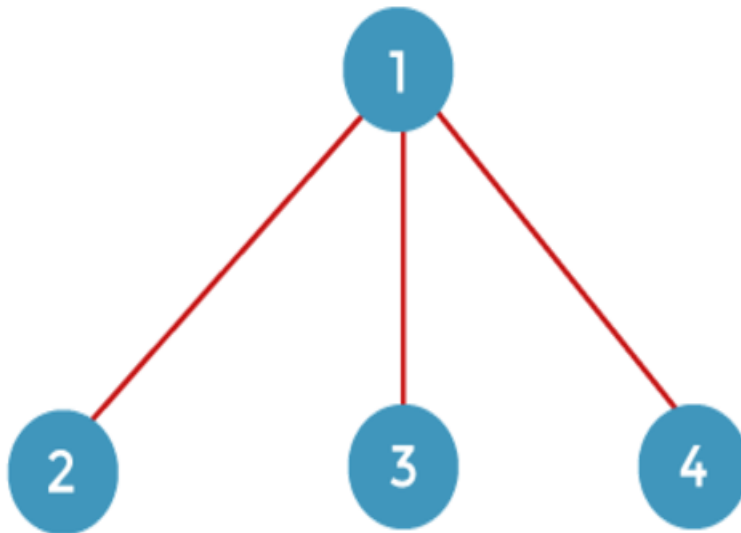
	1	2	3	4	5
1	∞	20	30	10	11
2	15	∞	30	10	11
3	3	5	∞	2	4
4	19	6	18	∞	3
5	16	4	7	16	∞

Let's first understand the approach then we solve the above problem.

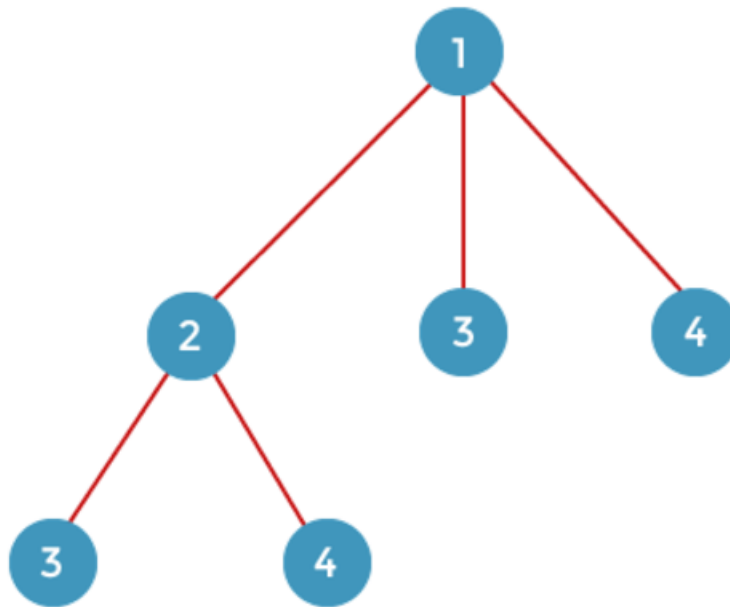
The graph is given below, which has four vertices:



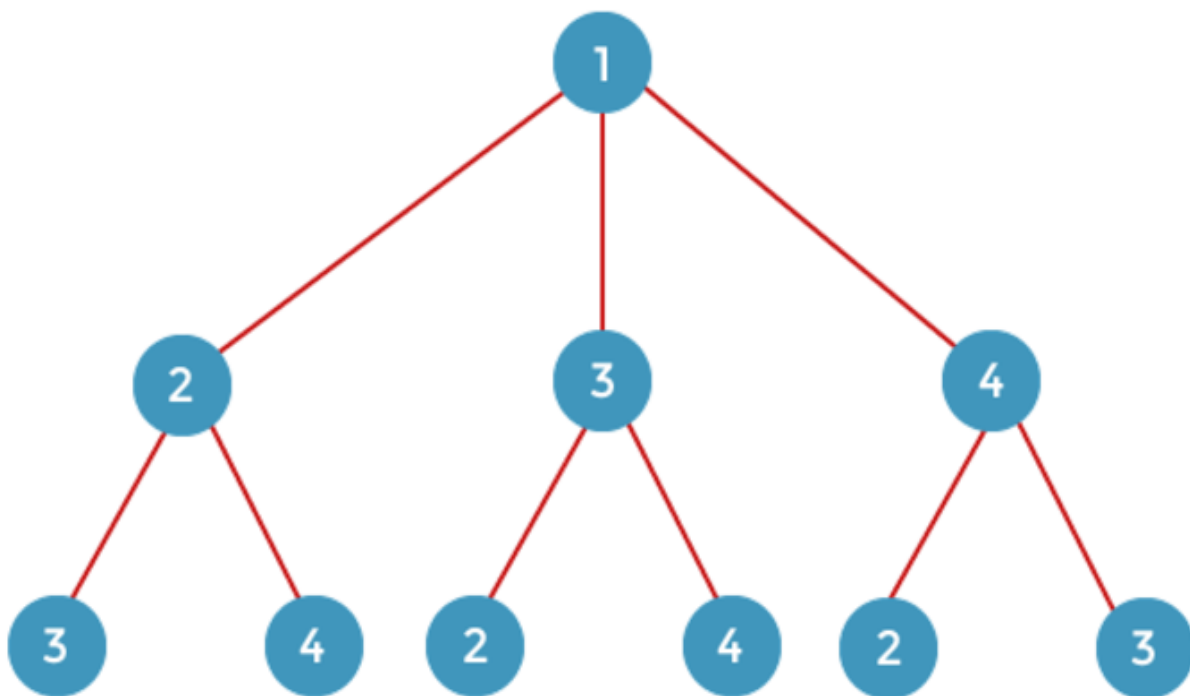
Suppose we start travelling from vertex 1 and return back to vertex 1. There are various ways to travel through all the vertices and returns to vertex 1. We require some tools that can be used to minimize the overall cost. To solve this problem, we make a state space tree. From the starting vertex 1, we can go to either vertices 2, 3, or 4, as shown in the below diagram.



From vertex 2, we can go either to vertex 3 or 4. If we consider vertex 3, we move to the remaining vertex, i.e., 4. If we consider the vertex 4 shown in the below diagram:



From vertex 4, we can go to the remaining vertices, i.e., 2 or 3. If we consider vertex 2, then we move to the remaining vertex, i.e., 3, and if we consider the vertex 3, then we move to the remaining vertex, i.e., 2 shown in the below diagram:



```
#include <iostream>
```

```
#include <vector>
#include <climits>
#include <queue>
using namespace std;
// Define infinity as a large number
#define INF INT_MAX
// Structure to represent a node in the decision tree
struct Node {
    vector<bool> visited; // Tracks visited cities
    int level;           // Level in the decision tree
    int cost;            // Cost to reach this node
    int bound;           // Lower bound of the cost
    int city;            // Current city
    vector<int> path;     // Path taken so far
    // Comparator for priority queue (lower bound first)
    bool operator<(const Node& other) const {
        return bound > other.bound;
    }
};
// Function to calculate the lower bound for a node
int calculateBound(Node& node, const vector<vector<int>>& costMatrix, int n) {
    int bound = node.cost;
    // Add the minimum outgoing cost for each unvisited city
    for (int i = 0; i < n; i++) {
        if (!node.visited[i]) {
            int minCost = INF;
            for (int j = 0; j < n; j++) {
                if (i != j && !node.visited[j]) {
                    minCost = min(minCost, costMatrix[i][j]);
                }
            }
        }
    }
}
```

```
        if (minCost != INF) {
            bound += minCost;
        }
    }
}

return bound;
}

// Function to solve the TSP using Branch and Bound
int tspBranchAndBound(const vector<vector<int>>& costMatrix, int n) {
    priority_queue<Node> pq; // Min-heap for best-first search
    Node root;
    root.visited = vector<bool>(n, false);
    root.level = 0;
    root.cost = 0;
    root.city = 0;
    root.path.push_back(0);
    root.visited[0] = true;
    // Calculate the bound for the root node
    root.bound = calculateBound(root, costMatrix, n);
    pq.push(root);
    int bestCost = INF;
    vector<int> bestPath;
    // Branch and Bound process
    while (!pq.empty()) {
        Node curr = pq.top();
        pq.pop();
        // If the bound of the current node is greater than the best cost, prune it
        if (curr.bound >= bestCost) continue;
        // If at the last level, complete the tour
        if (curr.level == n - 1) {
            int finalCost = curr.cost + costMatrix[curr.city][0]; // Return to start
```

```

        if (finalCost < bestCost) {
            bestCost = finalCost;
            bestPath = curr.path;
            bestPath.push_back(0); // Add starting city to complete the tour
        }
        continue;
    }
    // Expand the current node
    for (int i = 0; i < n; i++) {
        if (!curr.visited[i] && costMatrix[curr.city][i] != INF) {
            Node child = curr;
            child.level = curr.level + 1;
            child.city = i;
            child.path.push_back(i);
            child.visited[i] = true;
            child.cost += costMatrix[curr.city][i];
            child.bound = calculateBound(child, costMatrix, n);
            // Add the child node if its bound is promising
            if (child.bound < bestCost) {
                pq.push(child);
            }
        }
    }
}

// Print the best path
cout << "Best Path: ";
for (int city : bestPath) {
    cout << city << " ";
}
cout << endl;
return bestCost;

```



```

    }
    // Main function
    int main() {
        // Example cost matrix
        vector<vector<int>> costMatrix = {
            {INF, 10, 15, 20},
            {10, INF, 35, 25},
            {15, 35, INF, 30},
            {20, 25, 30, INF}
        };
        int n = costMatrix.size();
        int bestCost = tspBranchAndBound(costMatrix, n);
        cout << "Minimum cost: " << bestCost << endl;
        return 0;
    }

```

Input Cost Matrix:

INF 10 15 20

10 INF 35 25

15 35 INF 30

20 25 30 INF

Output:

Best Path: 0 1 3 2 0 Minimum cost: 80

➤ **Greedy Algorithms- Optimal Merge Patterns:**

Let us consider the given files, f_1 , f_2 , f_3 , f_4 and f_5 with 20, 30, 10, 5 and 30 number of elements respectively.

If merge operations are performed according to the provided sequence, then

$M_1 = \text{merge } f_1 \text{ and } f_2 \Rightarrow 20 + 30 = 50$, $M_2 = \text{merge } M_1 \text{ and } f_3 \Rightarrow 50 + 10 = 60$

$M_3 = \text{merge } M_2 \text{ and } f_4 \Rightarrow 60 + 5 = 65$, $M_4 = \text{merge } M_3 \text{ and } f_5 \Rightarrow 65 + 30 = 95$

DSA-Unit-V

Hence, the total number of operations is

$$50 + 60 + 65 + 95 = 270$$

Sorting the numbers according to their size in an ascending order, we get the following sequence

f_4, f_3, f_1, f_2, f_5

Hence, merge operations can be performed on this sequence

$M_1 = \text{merge } f_4 \text{ and } f_3 \Rightarrow 5 + 10 = 15, M_2 = \text{merge } M_1 \text{ and } f_1 \Rightarrow 15 + 20 = 35$

$M_3 = \text{merge } M_2 \text{ and } f_2 \Rightarrow 35 + 30 = 65, M_4 = \text{merge } M_3 \text{ and } f_5 \Rightarrow 65 + 30 = 95$

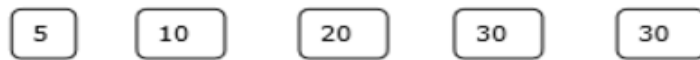
Therefore, the total number of operations is

$$15 + 35 + 65 + 95 = 210$$

Obviously, this is better than the previous one.

In this context, we are now going to solve the problem using this algorithm.

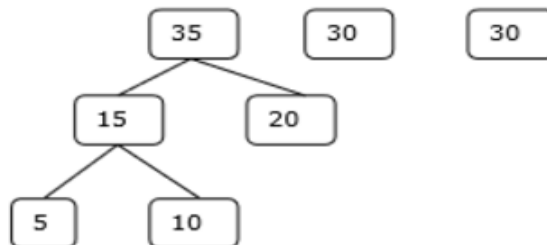
Initial Set



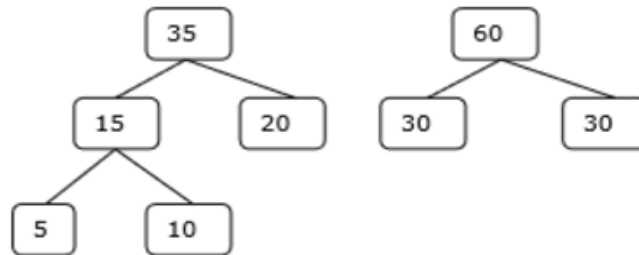
Step 1



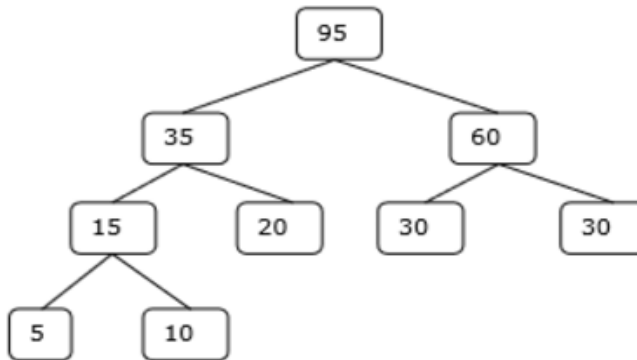
Step 2



Step 3



Step 4



Hence, the solution takes $15 + 35 + 60 + 95 = 205$ number of comparisons

```
#include <iostream>
```

```
#include <algorithm>
```

```
int optimalMerge(int files[], int n) {
```

```
    // Sort the files in ascending order
```

```
    for (int i = 0; i < n - 1; i++) {
```

```
        for (int j = 0; j < n - i - 1; j++) {
```

```
            if (files[j] > files[j + 1]) {
```

```
                std::swap(files[j], files[j + 1]);
```

```
            }
```

```
        }
```

```
    }
```

```
    int cost = 0;
```

```
    while (n > 1) {
```

```
// Merge the smallest two files
int mergedFileSize = files[0] + files[1];
cost += mergedFileSize;

// Replace the first file with the merged file size
files[0] = mergedFileSize;

// Shift the remaining files to the left
for (int i = 1; i < n - 1; i++) {
    files[i] = files[i + 1];
}

n--; // Reduce the number of files

// Sort the files again
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (files[j] > files[j + 1]) {
            std::swap(files[j], files[j + 1]);
        }
    }
}

return cost;
}

int main() {
    int files[] = {5, 10, 20, 30, 30};
    int n = sizeof(files) / sizeof(files[0]);
    int minCost = optimalMerge(files, n);
    std::cout << "Minimum cost of merging is: " << minCost << " Comparisons\n";
    return 0;
}
```

Output

Minimum cost of merging is: 205 Comparisons

➤ Huffman Coding | Greedy Algo-3:

Huffman coding is a lossless data compression algorithm. The idea is to assign variable-length codes to input characters, lengths of the assigned codes are based on the frequencies of corresponding characters.

The variable-length codes assigned to input characters are Prefix Codes, means the codes (bit sequences) are assigned in such a way that the code assigned to one character is not the prefix of code assigned to any other character. This is how Huffman Coding makes sure that there is no ambiguity when decoding the generated bitstream.

Algorithm:

The method which is used to construct optimal prefix code is called **Huffman coding**.

This algorithm builds a tree in bottom-up manner. We can denote this tree by **T**

Let, $|c|$ be number of leaves

$|c| - 1$ are number of operations required to merge the nodes. Q be the priority queue which can be used while constructing binary heap.

Algorithm Huffman (c)

```

{
     $n = |c|$ 
     $Q = c$ 
    for  $i \leftarrow 1$  to  $n-1$ 
    do
    {

         $temp \leftarrow \text{get\_node}()$ 
         $left[temp] \leftarrow \text{Get\_min}(Q)$ 
         $right[temp] \leftarrow \text{Get\_Min}(Q)$ 
         $a = left[temp]$ 
         $b = right[temp]$ 
         $F[temp] \leftarrow f[a] + f[b]$ 
         $\text{insert}(Q, temp)$ 
    }
    return  $\text{Get\_min}(Q)$ 
}

```

Steps to build Huffman Tree

Input is an array of unique characters along with their frequency of occurrences and output is Huffman Tree.

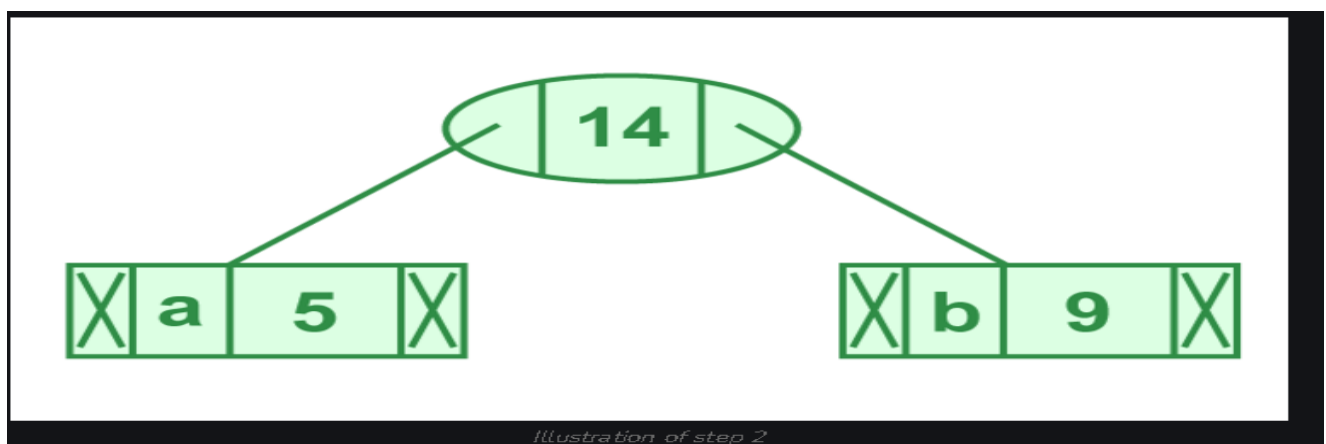
1. Create a leaf node for each unique character and build a min heap of all leaf nodes (Min Heap is used as a priority queue. The value of frequency field is used to compare two nodes in min heap. Initially, the least frequent character is at root)
2. Extract two nodes with the minimum frequency from the min heap.
3. Create a new internal node with a frequency equal to the sum of the two nodes frequencies. Make the first extracted node as its left child and the other extracted node as its right child. Add this node to the min heap.
4. Repeat steps#2 and #3 until the heap contains only one node. The remaining node is the root node and the tree is complete.

character Frequency

a	5
b	9
c	12
d	13
e	16
f	45

Step 1. Build a min heap that contains 6 nodes where each node represents root of a tree with single node.

Step 2 Extract two minimum frequency nodes from min heap. Add a new internal node with frequency $5 + 9 = 14$.



DSA-Unit-V

Now min heap contains 5 nodes where 4 nodes are roots of trees with single element each, and one heap node is root of tree with 3 elements

character	Frequency
c	12
d	13
Internal Node	14
e	16
f	45

Step 3: Extract two minimum frequency nodes from heap. Add a new internal node with frequency $12 + 13 = 25$

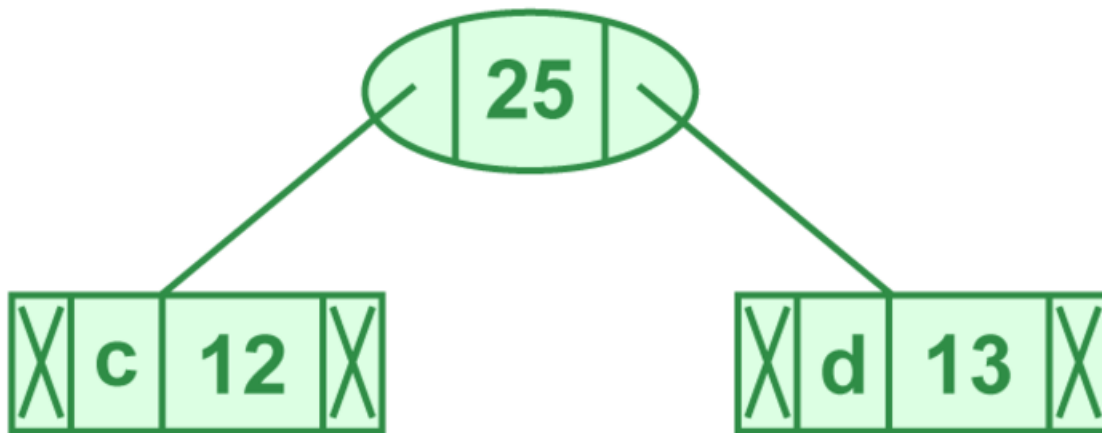


Illustration of step 3

character	Frequency
Internal Node	14
e	16
Internal Node	25
f	45

Step 4: Extract two minimum frequency nodes. Add a new internal node with frequency $14 + 16 = 30$

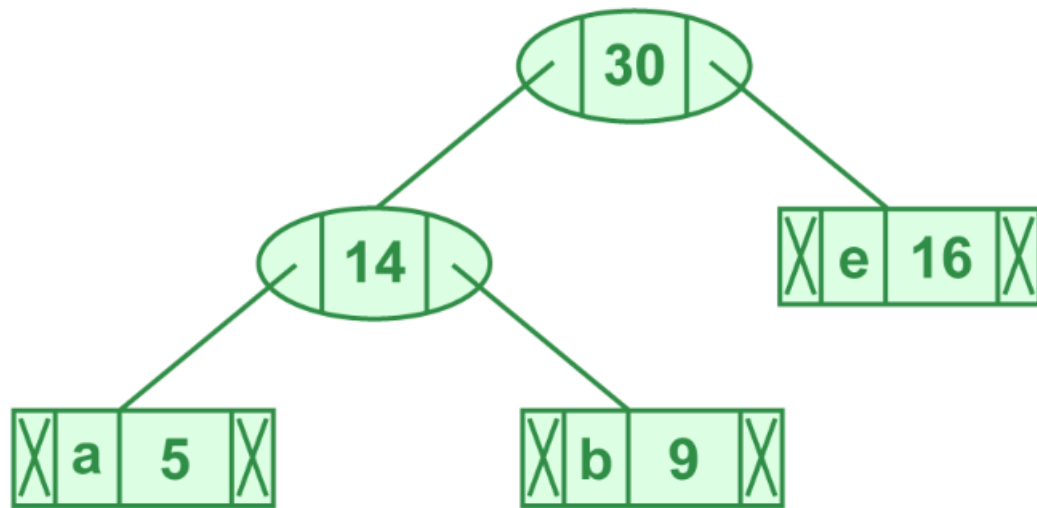


Illustration of step 4

Now min heap contains 2 nodes.

character	Frequency
f	45
Internal Node	55

Step 6: Extract two minimum frequency nodes. Add a new internal node with frequency $45 + 55 = 100$

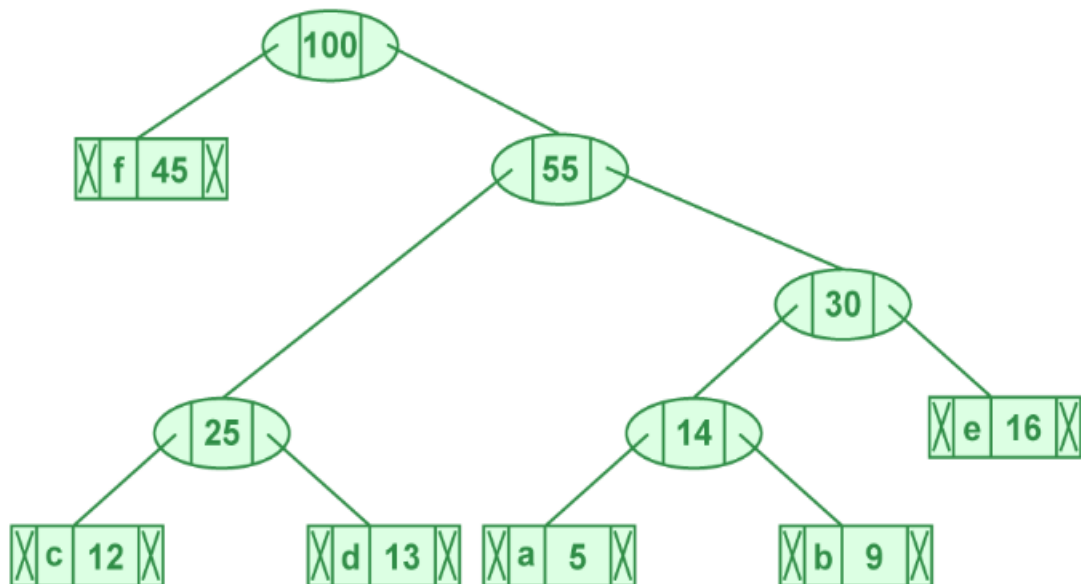


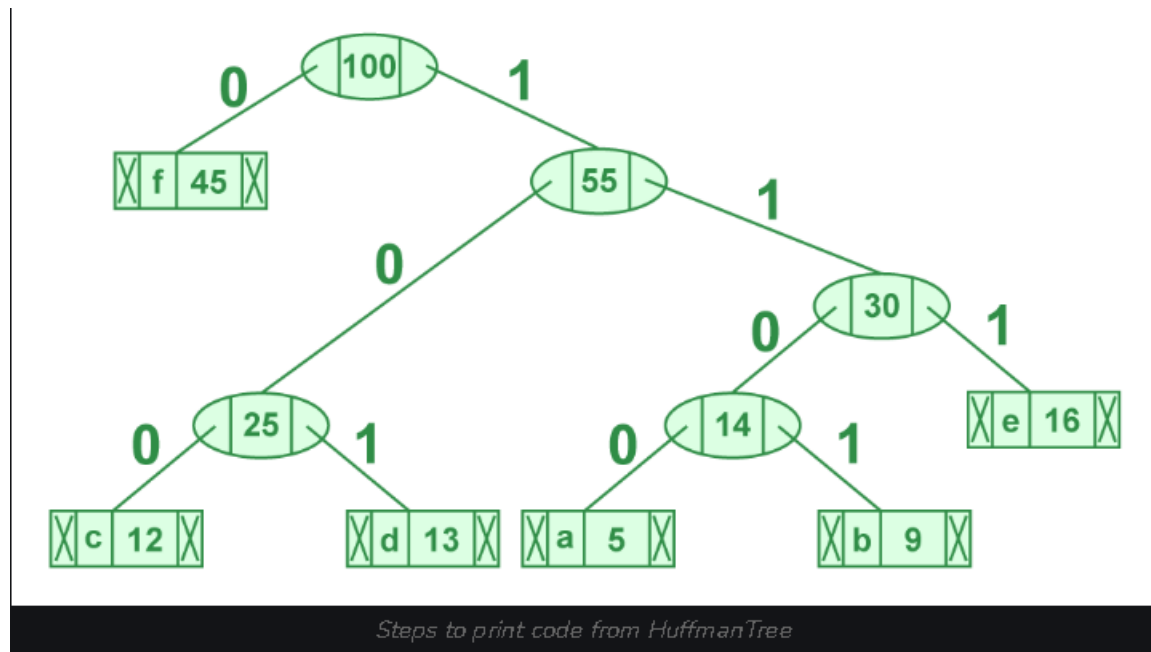
Illustration of step 6

Now min heap contains only one node.

character	Frequency
Internal Node	100

Steps to print codes from Huffman Tree:

Traverse the tree formed starting from the root. Maintain an auxiliary array. While moving to the left child, write 0 to the array. While moving to the right child, write 1 to the array. Print the array when a leaf node is encountered.



➤ **Knapsack Problem:**

The **Knapsack problem** is an example of the combinational optimization problem. This problem is also commonly known as the “**Rucksack Problem**”.

Types of Knapsack Problem:

The knapsack problem can be classified into the following types:

1. Fractional Knapsack Problem

The Fractional Knapsack problem can be defined as follows:

Given the weights and values of **N** items, put these items in a knapsack of capacity **W** to get the maximum total value in the knapsack. In Fractional Knapsack, we can break items for maximizing the total value of the knapsack.

Example:

Consider the example: `arr[] = {{100, 20}, {60, 10}, {120, 30}}`, **W = 50**.

Sorting: Initially sort the array based on the profit/weight ratio. The sorted array will be `{{60, 10}, {100, 20}, {120, 30}}`.

Iteration:

- For **i = 0**, weight = 10 which is less than **W**. So, add this element in the knapsack. **profit = 60** and remaining **W = 50 – 10 = 40**.
- For **i = 1**, weight = 20 which is less than **W**. So, add this element too. **profit = 60 + 100 = 160** and remaining **W = 40 – 20 = 20**.
- For **i = 2**, weight = 30 is greater than **W**. So, add 20/30 fraction = **2/3** fraction of the element. Therefore **profit = 2/3 * 120 + 160 = 80 + 160 = 240** and remaining **W** becomes **0**.

So, the final profit becomes **240** for **W = 50**.

// C++ program to solve fractional Knapsack Problem

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
// Structure for an item which stores weight and
```

```
// corresponding value of Item
```

```
struct Item {
```

```
    int profit, weight;
```

```
    // Constructor
```

```
    Item(int profit, int weight)
```

```
    {
```

```
        this->profit = profit;
```

```
        this->weight = weight;
```

```
    }
```

```
};
```

```
// Comparison function to sort Item
```

```
// according to profit/weight ratio
```

```
static bool cmp(struct Item a, struct Item b)
```

```

{
    double r1 = (double)a.profit / (double)a.weight;
    double r2 = (double)b.profit / (double)b.weight;
    return r1 > r2;
}
// Main greedy function to solve problem
double fractionalKnapsack(int W, struct Item arr[], int N)
{
    // Sorting Item on basis of ratio
    sort(arr, arr + N, cmp);
    double finalvalue = 0.0;
    // Looping through all items
    for (int i = 0; i < N; i++) {
        // If adding Item won't overflow,
        // add it completely
        if (arr[i].weight <= W) {
            W -= arr[i].weight;
            finalvalue += arr[i].profit;
        }
        // If we can't add current Item,
        // add fractional part of it
        else {
            finalvalue
            += arr[i].profit
            * ((double)W / (double)arr[i].weight);
            break;
        }
    }
    // Returning final value
    return finalvalue;
}

```

```
// Driver code
int main()
{
    int W = 50;
    Item arr[] = { { 60, 10 }, { 100, 20 }, { 120, 30 } };
    int N = sizeof(arr) / sizeof(arr[0]);
    // Function call
    cout << fractionalKnapsack(W, arr, N);
    return 0;
}
```

Output

240

2. 0/1 Knapsack Problem

We are given N items where each item has some weight (w_i) and value (v_i) associated with it. We are also given a bag with capacity W. The target is to put the items into the bag such that the sum of values associated with them is the maximum possible.

Note that here we can either put an item completely into the bag or cannot put it at all.

Mathematically the problem can be expressed as:

$$\text{Maximize } \sum_{i=1}^N v_i x_i \quad \text{subject to } \sum_{i=1}^N w_i x_i \leq W \quad \text{and } x_i \in \{0, 1\}$$

Example:

Input: N = 3, W = 4, profit [] = {1, 2, 3}, weight [] = {4, 5, 1}

Output: 3

Explanation: There are two items which have weight less than or equal to 4. If we select the item with weight 4, the possible profit is 1. And if we select the item with weight 1, the possible profit is 3. So, the maximum possible profit is 3. Note that we cannot put both the items with weight 4 and 1 together as the capacity of the bag is 4.

Input: N = 3, W = 3, profit [] = {1, 2, 3}, weight [] = {4, 5, 6}

Output: 0

```
/* A Naive recursive implementation of
0-1 Knapsack problem */
#include <bits/stdc++.h>
using namespace std;
// Returns the maximum value that
// can be put in a knapsack of capacity W
int knapSack(int W, int wt[], int val[], int n)
{
    // Base Case
    if (n == 0 || W == 0)
        return 0;
    // If weight of the nth item is more
    // than Knapsack capacity W, then
    // this item cannot be included
    // in the optimal solution
    if (wt[n - 1] > W)
        return knapSack(W, wt, val, n - 1);
    // Return the maximum of two cases:
    // (1) nth item included
    // (2) not included
    else
        return max(knapSack(W, wt, val, n - 1),
            val[n - 1] + knapSack(W - wt[n - 1], wt, val, n - 1));
}
// Driver code
int main()
{
    int profit[] = { 60, 100, 120 };
    int weight[] = { 10, 20, 30 };
    int W = 50;
    int n = sizeof(profit) / sizeof(profit[0]);
```

```
cout << knapSack(W, weight, profit, n);
return 0;
}
```

Output

220

Sr. No	0/1 knapsack problem	Fractional knapsack problem
1.	The 0/1 knapsack problem is solved using dynamic programming approach.	Fractional knapsack problem is solved using a greedy approach.
2.	In the 0/1 knapsack problem, we are not allowed to break items.	Fractional knapsack problem, we can break items for maximizing the total value of the knapsack.
3.	0/1 knapsack problem, finds a most valuable subset item with a total value less than equal to weight.	In the fractional knapsack problem, finds a most valuable subset item with a total value equal to the weight if the total weight of items is more than or equal to the knapsack capacity.
4.	In the 0/1 knapsack problem we can take objects in an integer value.	In the fractional knapsack problem, we can take objects in fractions in floating points.

3. Bounded Knapsack Problem

Given **N** items, each item having a given weight **w_i** and a value **v_i**, the task is to maximize the value by selecting a maximum of **K** items adding up to a maximum weight **W**

Mathematically the problem can be expressed as:

Maximize $\sum_{i=1}^N v_i x_i$ subject to $\sum_{i=1}^N w_i x_i \leq W$ and $x_i \in \{0, 1, \dots, K\}$

Example:

Input: N = 5, P[] = {2, 7, 1, 5, 3}, C[] = {2, 5, 2, 3, 4}, W = 8, K = 2.

Output: 12

Explanation:

Here, the maximum possible profit is when we take 2 items: item2 (P[1] = 7 and C[1] = 5) and item4 (P[3] = 5 and C[3] = 3).

Hence, maximum profit = 7 + 5 = 12

Input: N = 5, P[] = {2, 7, 1, 5, 3}, C[] = {2, 5, 2, 3, 4}, W = 1, K = 2

Output: 0

Explanation: All weights are greater than 1. Hence, no item can be picked.

```
// C++ code for the extended
// Knapsack Approach
#include <bits/stdc++.h>
using namespace std;
// To store the dp values
vector<vector<vector<int>>> dp;
int maxProfit(int profit[], int weight[], int n, int max_W,
              int max_E)
{
    // for each element given
    for (int i = 1; i <= n; i++) {
        // For each possible
        // weight value
        for (int j = 1; j <= max_W; j++) {
            // For each case where
            // the total elements are
            // less than the constraint
            for (int k = 1; k <= max_E; k++) {
                // To ensure that we dont
                // go out of the array
                if (j >= weight[i - 1]) {
```

```

        dp[i][j][k] = max(
            dp[i - 1][j][k],
            dp[i - 1][j - weight[i - 1]][k - 1]
            + profit[i - 1]);
    }
    else {
        dp[i][j][k] = dp[i - 1][j][k];
    }
}
}
}
return dp[n][max_W][max_E];
}
// Driver Code
int main()
{
    int n = 5;
    int profit[] = { 2, 7, 1, 5, 3 };
    int weight[] = { 2, 5, 2, 3, 4 };
    int max_weight = 8;
    int max_element = 2;
    dp = vector<vector<vector<int>>>(
        n + 1, vector<vector<int>>(
            max_weight + 1,
            vector<int>(max_element + 1, 0)));
    cout << maxProfit(profit, weight, n, max_weight,
        max_element)
        << "\n";
    return 0;
}

```

Output : 12

4. Unbounded Knapsack Problem

Given a knapsack weight W and a set of N items with certain value v_i and weight w_i , we need to calculate the maximum amount that could make up this quantity exactly. This is different from **0/1 Knapsack problem**, here we are allowed to use an unlimited number of instances of an item.

Mathematically the problem can be expressed as:

$$\text{Maximize } \sum_{i=1}^N v_i x_i \text{ subject to } \sum_{i=1}^N w_i x_i \leq W \text{ and } x_i \in \mathbb{Z} \text{ and } x_i \geq 0.$$

Example:

Input: val[] = {14, 27, 44, 19}, wt[] = {6, 7, 9, 8}, W = 50

Output: 244.444

```
// C++ implementation of the approach
#include <bits/stdc++.h>
using namespace std;
// Function to return the maximum required value
float knapSack(int W, float wt[], float val[], int n)
{
    // maxratio will store the maximum value to weight
    // ratio we can have for any item and maxindex
    // will store the index of that element
    float maxratio = INT_MIN;
    int maxindex = 0;

    // Find the maximum ratio
    for (int i = 0; i < n; i++) {
        if ((val[i] / wt[i]) > maxratio) {
            maxratio = (val[i] / wt[i]);
            maxindex = i;
        }
    }
}
```

```

        // The item with the maximum value to
        // weight ratio will be put into
        // the knapsack repeatedly until full
        return (W * maxratio);
    }
    // Driver code
    int main()
    {
        float val[] = { 14, 27, 44, 19 };
        float wt[] = { 6, 7, 9, 8 };
        int n = sizeof(val) / sizeof(val[0]);
        int W = 50;
        cout << knapSack(W, wt, val, n);
        return 0;
    }

```

➤ **Matrix Chain Multiplication Problem using Dynamic Programming:**

The Matrix Chain Multiplication problem is a classic optimization problem solved using Dynamic Programming (DP). The goal is to determine the most efficient way to multiply a chain of matrices, minimizing the total number of scalar multiplications.

Given a sequence of matrices $A_1, A_2, \dots, A_n$, where the dimensions of A_i are $p[i-1] \times p[i]$, determine the minimum number of scalar multiplications needed to compute the product $A_1 A_2 \dots A_n$.

Key Idea:

- Use a table to store the minimum number of multiplications required for each subproblem.
- Solve smaller subproblems first and use their results to solve larger subproblems.

Steps:

1. Define $dp[i][j]$ as the minimum number of multiplications needed to multiply matrices A_i to A_j .

2. Base case: $dp[i][i]=0$ (single matrix requires no multiplication).
3. Recurrence relation:

$$dp[i][j] = \min_{k=i}^{j-1} (dp[i][k] + dp[k+1][j] + p[i-1] \cdot p[k] \cdot p[j])$$

- Split the matrices at every possible k between i and j .
 - Add the cost of multiplying the resulting matrices.
4. Fill the DP table iteratively, starting with smaller matrix chains and expanding to larger ones.

```
#include <iostream>
#include <climits>

using namespace std;

// Function to find the minimum multiplication cost
int matrixChainMultiplication(int p[], int n) {
    // dp[i][j] will store the minimum number of multiplications for matrices i to j
    int dp[n][n];

    // Base case: Cost is zero when multiplying one matrix
    for (int i = 1; i < n; i++)
        dp[i][i] = 0;

    // l is chain length (from 2 to n)
    for (int l = 2; l < n; l++) {
        for (int i = 1; i < n - l + 1; i++) {
            int j = i + l - 1;
            dp[i][j] = INT_MAX;
            // Try all possible splits
            for (int k = i; k < j; k++) {
                int cost = dp[i][k] + dp[k+1][j] + p[i-1] * p[k] * p[j];
                dp[i][j] = min(dp[i][j], cost);
            }
        }
    }
}
```

```
    }  
    // Minimum cost to multiply matrices from 1 to n-1  
    return dp[1][n-1];  
}  
int main() {  
    int n; // Number of matrices + 1  
    cout << "Enter the number of matrices: ";  
    cin >> n;  
    int p[n];  
    cout << "Enter the dimensions array: ";  
    for (int i = 0; i < n; i++) {  
        cin >> p[i];  
    }  
    cout << "Minimum number of multiplications: " << matrixChainMultiplication(p,  
n) << endl;  
    return 0;  
}
```

Input:

Enter the number of matrices: 4

Enter the dimensions array: 10 20 30 40 30

Output:

Minimum number of multiplications: 30000

Explanation:

- **Dimensions:** $A_1=10 \times 20$, $A_2=20 \times 30$, $A_3=30 \times 40$, $A_4=40 \times 30$.
- **Optimal parenthesization:** $(A_1 \times A_2) \times (A_3 \times A_4)$ $(A_1 \times A_2) \times (A_3 \times A_4)$.
- **Minimum cost = 30000 scalar multiplications.**